

OpenSSL PKI Study Notes

Root CA · Intermediate CA · Certificate Chaining · Extensions

airlegit Fintech Platform — 3-Tier PKI Reference

1. OpenSSL Config — Section Headers

OpenSSL config files use INI-style section delimiters. Each command reads a specific section, which chains to others via pointer keys.

```
[ ca ]           → entry point for openssl ca command
[ CA_default ]  → actual CA operational parameters
[ policy_strict ]→ DN field validation rules
[ req ]         → entry point for openssl req command
[ req_distinguished_name ] → DN prompts and defaults
[ v3_ca ]       → Root CA certificate extensions
[ v3_intermediate_ca ] → Intermediate CA extensions
[ crl_ext ]     → CRL extensions
```

The pointer chain:

```
[ ca ]
    ■■■ default_ca = CA_default ■■■ [ CA_default ]
                                ■■■ policy = policy_strict
                                ■■■ x509_extensions = v3_intermediate_ca
                                ■■■ crl_extensions = crl_ext

[ req ]
    ■■■ distinguished_name = req_distinguished_name
    ■■■ x509_extensions = v3_ca
```

Command → Section mapping:

Command	Reads Section	Pulls In
openssl ca	[ca]	CA_default → policy_strict → v3_intermediate_ca
openssl req -x509	[req]	req_distinguished_name → v3_ca
openssl req (CSR)	[req]	req_distinguished_name
openssl ca -gencrl	[CA_default]	crl_ext

2. [CA_default] — Key Parameters

Parameter	Value	Purpose
database	index.dat	Flat-file ledger of all issued/revoked certs
serial	serial.dat	Hex counter — auto-incremented per cert issued
default_days	3650	~10 years validity for CA-signed certs
default_crl_days	30	CRL validity period before regeneration needed
unique_subject	no	Allow multiple valid certs with same Subject DN (cert rotation)
preserve	no	Reorder Subject DN fields to match CA's own order
email_in_dn	no	Exclude email from Subject DN (less PII exposure)

3. [policy_strict] — DN Validation

Enforced when Root CA signs an Intermediate CA CSR. Compares CSR Subject DN against CA's own cert field by field.

Directive	Meaning	Failure Behaviour
supplied	Field must exist in CSR (any value)	OpenSSL rejects CSR
match	Field must exist AND match CA's own value exactly	OpenSSL rejects CSR
optional	Field may be absent, any value accepted	Never causes rejection

In your config: **organizationName = match** → Intermediate CA CSR must carry exactly **O=prn**.
Case-sensitive.

policy_strict vs policy_loose:

```
Root CA  ■■(policy_strict)■■■ Intermediate CA CSR
          O= must match, C= must exist

Intermediate CA  ■■(policy_loose)■■■ Leaf cert CSR
                  O= doesn't matter, just CN= required
```

4. openssl req -x509 vs openssl req (CSR only)

Step	openssl req (CSR)	openssl req -x509
Generate keypair	■	■
Build Subject DN	■	■
Apply x509 extensions	■	■ from x509_extensions
Sign with CA key	■ needs openssl ca	■ signs itself
Output	.csr file	.cer file
Validity period	None	-days flag
Trusted by default	■	■ must be manually trusted

Only the **Root CA** uses **-x509** — there is no higher authority to sign its CSR. The Root is a trust anchor, trusted because you explicitly install it into the trust store.

In airlegit PKI flow:

```
Step 1 – Root CA (self-signed)
  openssl req -new -x509 -config rootca.cnf -key rootca.key -out rootca.cer -days 3650

Step 2 – Intermediate CA (CSR only)
  openssl req -new -config intermediateca.cnf -key intermediateca.key -out intermediateca.csr

Step 3 – Root signs Intermediate
  openssl ca -config rootca.cnf -extensions v3_intermediate_ca -in intermediateca.csr -out intermediateca.cer

Step 4 – Kafka broker (CSR only)
  openssl req -new -config kafka.cnf -key kafka.key -out kafka.csr

Step 5 – Intermediate signs broker
  openssl ca -config intermediateca.cnf -extensions v3_server -in kafka.csr -out kafka.cer
```

5. [v3_ca] vs [v3_intermediate_ca] Extensions

basicConstraints

```
Root CA:      basicConstraints = critical, CA:true, pathlen:2
Intermediate CA: basicConstraints = critical, CA:true, pathlen:0
```

CA:true — marks this cert as a Certificate Authority. Both Root and Intermediate are CAs — CA:true applies to both. Leaf certs (Kafka brokers, microservices) do NOT have CA:true.

critical — any client that doesn't understand this extension must reject the cert outright.

pathlen — counts how many more CA certificates can appear below this point in the chain:

pathlen value	Meaning
pathlen:2 (Root CA)	Up to 2 more CA certs allowed below — gives headroom for future tiers
pathlen:0 (Intermediate CA)	Zero more CAs allowed below — can only sign leaf certs
not set (Leaf cert)	Not a CA, signs nothing

■ *pathlen counts CA hops remaining, NOT total certs that can be generated.*

keyUsage

```
Root CA:      keyUsage = critical, keyCertSign, cRLSign
Intermediate CA: keyUsage = critical, keyCertSign, cRLSign, digitalSignature
```

Usage	Root CA	Intermediate CA	Purpose
keyCertSign	■	■	Can sign certificates
cRLSign	■	■	Can sign CRLs
digitalSignature	■	■	Can sign arbitrary data — needed for OCSP responder

Root CA omits **digitalSignature** intentionally — its key should only ever be used for signing certs and CRLs. Kept minimal and ideally offline.

6. subjectKeyIdentifier & authorityKeyIdentifier

These two fields are the mechanical glue of certificate chaining.

Field	Purpose	Set by
subjectKeyIdentifier (SKI)	Hash of THIS cert's public key — my own fingerprint	this cert itself
authorityKeyIdentifier (AKI)	Hash of SIGNER's public key — points to issuer's SKI	the CA that signed this cert

How the chain is built:

```
Root CA cert
SKI: AB:CD:EF ← hash of Root's public key
AKI: AB:CD:EF ← self-signed, points to itself

Intermediate CA cert
SKI: 11:22:33 ← hash of Intermediate's public key
AKI: AB:CD:EF ← Root signed me (Root's SKI)

Kafka broker cert
SKI: 99:88:77 ← hash of broker's public key
AKI: 11:22:33 ← Intermediate signed me (Intermediate's SKI)
```

Chain walking (client verification):

```
broker AKI: 11:22:33
  ■■■■ find cert where SKI = 11:22:33 → Intermediate CA ■
    ■■■■ Intermediate AKI: AB:CD:EF
      ■■■■ find cert where SKI = AB:CD:EF → Root CA ■
        ■■■■ Root AKI = Root SKI → self-signed, STOP
          ■■■■ in trust store? ■ → chain valid ■
```

Why not just use DN name matching? Two CAs can have identical DNs (e.g. during CA rotation/renewal) but will always have different public keys — so SKI/AKI matching is always unambiguous.

authorityKeyIdentifier = keyid:always,issuer

Component	Meaning
keyid	Copy issuer's SKI into this cert's AKI
keyid:always	Mandatory — throw error if issuer has no SKI (don't silently skip)
issuer	Also embed issuer's full DN + serial as fallback identifier

■ `[ crl_ext ]` uses only `keyid:always` (no `,issuer`) — CRLs are simpler, key fingerprint alone is sufficient.

7. crlDistributionPoints & authorityInfoAccess

crlDistributionPoints (CDP)

Tells clients where to fetch the CRL to check if a cert has been revoked.

```
crlDistributionPoints = URI:http://ca.local/rootca.crl
```

Rule: CDP in a cert points to the CRL of whoever **signed** that cert (one level up).

Cert	CDP points to	Because
Intermediate CA cert	rootca.crl	Root CA signed the Intermediate — Root revokes it
Kafka broker cert	intermediateca.crl	Intermediate signed the broker — Intermediate revokes it
Root CA cert	No CDP needed	Nothing above Root to revoke it

■ In your current config, leaf cert extensions (in intermediateca.cnf) must use intermediateca.crl — not rootca.crl.

authorityInfoAccess (AIA)

Tells clients where to find additional info about the issuer.

Access Method	Purpose
caIssuers;URI:...	URL to download the ISSUER's certificate — helps complete the chain
OCSP;URI:...	URL of OCSP responder for real-time revocation check (not configured yet)

caIssuers solves the problem where a server sends only its own cert without the intermediate. The client reads the AIA field, fetches the intermediate cert, and completes chain building. It walks one level at a time:

```
Client has broker cert
reads caIssuers → fetches intermediateca.cer
reads intermediateca.cer caIssuers → fetches rootca.cer
Root is self-signed → STOP → chain complete ■
```

CDP vs AIA — What's the Difference

Field	Fetches	Purpose
crlDistributionPoints	.crl file	Check if cert is REVOKED
authorityInfoAccess caIssuers	.cer file	Fetch issuer's cert to COMPLETE CHAIN
authorityInfoAccess OCSP	Live HTTP response	Real-time REVOCATION check

CRL vs OCSP

Method	How it works	Freshness	Use case
CRL	Client downloads entire revocation list	Stale, refreshed every 30 days	Internal PKI (your setup)
OCSP	Client asks responder: is serial X valid?	Always YES/NO	Public-facing PKI at scale

8. Interview Relevance — DevOps / SRE

Topic	Importance	Notes
mTLS — what, why, how	■■■ High	Core fintech SRE topic
Certificate chain verification	■■■ High	Understand SKI/AKI walk
Kubernetes cert management (cert-manager)	■■■ High	Practical K8s skill
Vault PKI secrets engine	■■■ High	You have real hands-on here
TLS troubleshooting (openssl s_client)	■■■ High	Day-to-day SRE skill
CRL vs OCSP conceptually	■■ Medium	Know the difference
OpenSSL config internals	■ Low	Backup depth — rarely asked directly
SKI/AKI internals	■ Low	Gives accuracy to your answers

Your strongest interview answer:

```
"I architected a 3-tier PKI from scratch using OpenSSL –
Root CA → Intermediate CA → leaf certs for Kafka brokers
and microservices, with mTLS enforced across the cluster."
```

Common Interview Questions

Question	Key Answer Points
What is mTLS vs TLS?	TLS = server proves identity. mTLS = both client AND server prove identity via certs.
What is a certificate chain?	SKI/AKI linked list from leaf → intermediate → root. Client walks up until trust store hit.
Root CA vs Intermediate CA?	Root = self-signed trust anchor, offline. Intermediate = online signing CA, pathlen:0.
What is a CSR?	Certificate Signing Request — contains public key + Subject DN. Submitted to CA for sig.
CRL vs OCSP?	CRL = downloaded list, can be stale. OCSP = real-time query, always current.
Cert rotation in Kubernetes?	cert-manager + Let's Encrypt or Vault PKI, automatic renewal before expiry.

9. Quick Reference — Mental Models

```
CA:true      = this cert belongs to a Certificate Authority
pathlen:N    = N more CA hops allowed below me
pathlen:0    = I can sign leaf certs but NO more CAs

SKI          = my own public key fingerprint
AKI          = my signer's public key fingerprint

CDP          = where to fetch revocation list (CRL)
caIssuers    = where to fetch my issuer's cert
OCSP         = where to check real-time revocation

supplied     = field must exist in CSR (any value)
match        = field must exist AND match CA's value exactly
optional     = field may be absent

openssl req -x509 = self-signed cert (Root CA only)
openssl req      = CSR only (Intermediate, leaf certs)
openssl ca       = CA signs a CSR → issues a cert
```

3-Tier PKI Chain — airlegit

```
Root CA (pathlen:2, CA:true, self-signed)
  ■ signs with policy_strict (0= must match)
  ▼
Intermediate CA (pathlen:0, CA:true, signed by Root)
  ■ signs with policy_loose
  ▼
Leaf Certs (CA:false, signed by Intermediate)
  ■■■ Kafka broker certs (serverAuth, clientAuth, SAN=DNS/IP)
  ■■■ Microservice certs (clientAuth, SAN=DNS)
```

Next Steps — Leaf Cert Extensions (Not Covered Yet)

Extension	Purpose
extendedKeyUsage = serverAuth	TLS server role — Kafka brokers
extendedKeyUsage = clientAuth	TLS client role — microservices doing mTLS
subjectAltName = DNS:..., IP:...	Mandatory for modern TLS — CN alone is ignored by clients